Neural Operators for Ultrasound Propagation

1 Introduction

I am an undergraduate who specializes in digital signal processing. I have the unique opportunity to help a professor of mine create a class in Applied DSP, talking about real-time DSP systems. To make this class, I am developing a series of simulations where I task the student to develop a DSP pipeline.

In this specific case, I want the student to learn all about Beamforming and Sonar through Ultrasound simulations. For the simulator, the student will generate a transmission pulse and a steering vector, which are going straight to the simulated ultrasound transducer. The student will then receive a received transmission wave for each probe on the ultrasound transducer. For the purposes of this class, I limit the number to 64. This process will also be done in real time and I ideally want the student to deal with the nonlinearities of ultrasound propagation and acoustic propagation in a fluid-filled medium.

The current state-of-the-art ultrasound propagation simulators use k-space pseudo-spectral methods. The most popular solver is k-Wave.

Though the simulator is highly accurate, it is computationally expensive. Nonetheless, it can model arbitrary heterogeneous media, frequency-dependent attenuation, and nonlinear harmonic generation.

Still, they are too computationally expensive for real-time applications. For example, a single forward simulation for a 64-element Uniform Linear Array (ULA) that is simulated with the k-Wave iterative structure requires a high-density spatial grid and takes about 3–4 minutes to complete. For the purposes of this assignment, that is not possible. We are hoping for latency on the order of 100ms rather than 100s.

Thus, this project aims to develop a deep neural network surrogate model to bypass this computational bottleneck. We do this by mapping the spatial parameters of the medium and the TX RF pulse directly to the received radio-frequency (RX) channel data.

We want to start by mathematically grounding our approach using the rigorous nonlinear physical reference model, classical LTI spatial interpolation, and finally, Neural Operators.

2 Problem Formulation

Let's start by formulating the problem. Let's first look at the physical environment that we propagate the ultrasound waves through.

As this is a sort of toy example for teaching purposes, it is sufficient for me to model the acoustic medium as static, heterogeneous, and 2D,

$$\Omega \subset \mathbb{R}^2$$

with spatial coordinates

$$(x, z) \in \Omega$$

Here, x represents the lateral direction and z represents the axial depth.

I use a 2D model because ultrasound propagation depends heavily on how the medium changes across space. At the end of the day, we care about teaching beamforming, so having two dimensions is critical. Adding 3 dimensions makes the problem much harder to solve and much more complicated without any additional learning.

Now let's start by looking at how a wave can interact with its physical domain. These methods include scattering, refraction, phase aberration, and acoustic shadowing.

All of these depend on spatial heterogeneity across the imaging plane. Because of this, using a simple 1D representation of the medium is not sufficient even if we didn't focus on beamforming applications. So using 2D spatial maps is critical to describe the physical properties of the tissue.

2.1 Medium Parameterization

We need 3 fields to properly parameterize the acoustic medium.

First, I define the sound speed map

$$c(x, z)$$

This field specifies the local speed of sound at every point in the medium. The units are meters per second.

Next, I define the density map

$$\rho(x, z)$$

This map defines the local mass density of the medium in kg/m^3.

Finally, I introduce an inclusion mask

$$M(x, z) \in [0, 1]$$

This mask allows the simulator to define special regions in the domain. Using this mask, I can create structures such as rigid reflectors, anechoic cysts, and absorbing boundaries.

These regions behave differently from the background medium and introduce strong scattering effects. This mask is why the problem becomes so hard.

2.2 Acoustic Impedance

Once the sound speed and density fields are defined, you can then compute acoustic impedance

$$Z(x, z) = \rho(x, z)c(x, z)$$

Acoustic impedance determines how strongly an acoustic wave reflects when it encounters a boundary between two materials.

When the impedance changes abruptly across space, the wave scatters. As a result, spatial variations in $Z(x, z)$ determine the local scattering potential of the medium. This is what shows up as "speckle" in the final ultrasound image. It also plays a large part in shaping the received RX signal.

2.3 Transducer Geometry

Now, let's talk about the actual ultrasound probe or transducer.

For the simulator, I use a 2D 64-element Uniform Linear Array (ULA). Since we are only looking at this in 2D for the sake of this experiment, this means that the depth coordinate z_i is fixed and the elements vary laterally along x.

Thus it is a 64x1 array in the xy plane. Each element in this array can both transmit and receive acoustic waves.

This configuration is pretty realistic to what you would see in an actual ultrasound probe. The only difference is that we are not considering the y plane in our example.

2.4 Discretized Representation

So far I have described the medium as a continuous spatial domain. But, both numerical simulation and training data for machine learning naturally deal with discrete information.

Thus, we need to discretize the spatial domain using a Cartesian grid of size

$$N_x \times N_z$$

which produces tensor-valued maps

$$c \in \mathbb{R}^{N_x \times N_z}, \qquad \rho \in \mathbb{R}^{N_x \times N_z}, \qquad M \in [0, 1]^{N_x \times N_z}$$

These maps fully describe the physical state of the acoustic medium.

This representation is used by pseudo-spectral solvers such as k-Wave, linear approximations of the k-Wave simulation, and neural surrogate models that attempt to predict the received RF channel data directly from the spatial configuration of the medium and the transmit excitation.

3 Nonlinear Acoustic Propagation

To generate what we use as our accurate ground-truth simulations, I use the k-Wave pseudo-spectral solver.

This solver numerically integrates the nonlinear acoustic wave equations in time over the 2D spatial domain (x, z).

It simulates acoustic propagation by tracking three physical state variables that evolve in both space and time: the acoustic pressure field $p(x, z, t)$, the particle velocity field $u(x, z, t)$, and the density perturbation of the medium $\rho(x, z, t)$.

This state is enough to describe how the acoustic waves propagate through the tissue medium.

3.1 Governing Acoustic Equations

We can model how these fields evolve through coupled nonlinear acoustic equations.

First, we have the momentum conservation equation that describes how pressure gradients accelerate the fluid particles:

$$\frac{\partial u}{\partial t} = -\frac{1}{\rho_0(x, z)} \nabla p$$

Next, the mass conservation equation which describes how the density of the medium evolves as the fluid moves:

$$\frac{\partial \rho}{\partial t} = -\rho_0(x, z) \nabla \cdot \mathbf{u} - \mathbf{u} \cdot \nabla \rho_0(x, z)$$

And finally, we have the equation of state relates pressure to density and introduces nonlinear acoustic effects:

$$p = c_0^2(x, z) \left(\rho + \frac{B}{2A} \frac{\rho^2}{\rho_0(x, z)} \right) - L\{\mathbf{u}, p\}$$

Here, $c_0(x, z)$ is the background sound speed, $\rho_0(x, z)$ is the background density, B/A is the nonlinearity parameter, and $L\{u, p\}$ models frequency-dependent acoustic absorption and dispersion.

3.2 Physical Interpretation

Because the background medium parameters $c_0(x, z)$ and $\rho_0(x, z)$ vary across space, the acoustic wave does not propagate through a uniform environment.

Instead, as the wave travels through the domain it encounters spatial variations that cause scattering from impedance boundaries, refraction from sound-speed gradients, and phase distortion along heterogeneous paths.

This is what we later try to model.

Based on how the TX signal responds to these various effects, we can attempt to recover the properties of the medium it passed through by examining the 64 different RX waves and use that to image the material.

4 The Paradigm of Operator Learning for Ultrasound Propagation

4.1 Breaking the Iterative Bottleneck

Now that we have a baseline level of understanding about how accurate ultrasound simulation works and k-Wave, we are going to talk about the fundamental limitation of k-Wave which is speed.

k-Wave simulates acoustic propagation by stepping forward in time in very small increments. Then, at each timestep, the solver evaluates spatial gradients, updates the acoustic state and advances the system by a small time interval Δt. Furthermore, this timestep size is bounded by the Courant–Friedrichs–Lewy (CFL) stability condition, which means that to guarantee some degree of accuracy, we need to use small timesteps.

As the solver must march forward step by step, every point in the spatial domain needs to be updated sequentially. This ends up creating a hard limit for the speed of the simulation and even with an accelerated and optimized implementation that takes into account simplifying assumptions, the simulations take around $\sim$3 minutes per simulation.

For real-time DSP systems like the one I want to create for this class, this is about 3 orders of magnitude too slow.

So the next idea we have is what if instead of simulating the wave evolution by iterating in time, we try to learn the global forward mapping directly. This way we use the NN as a heuristic that somewhat accurately solves the global forward mapping.

What we really care about is the mapping that is TX RF signal $\rightarrow$ RX RF signal, where this mapping depends on medium configurations.

If a neural network can learn this mapping, then generating a new simulation becomes a single feed-forward inference pass, which would allow us to simulate ultrasound wave propagation in real time.

This is not unheard of. For example, for linear time-invariant systems, we can model this forward mapping as a convolution with a transfer function.

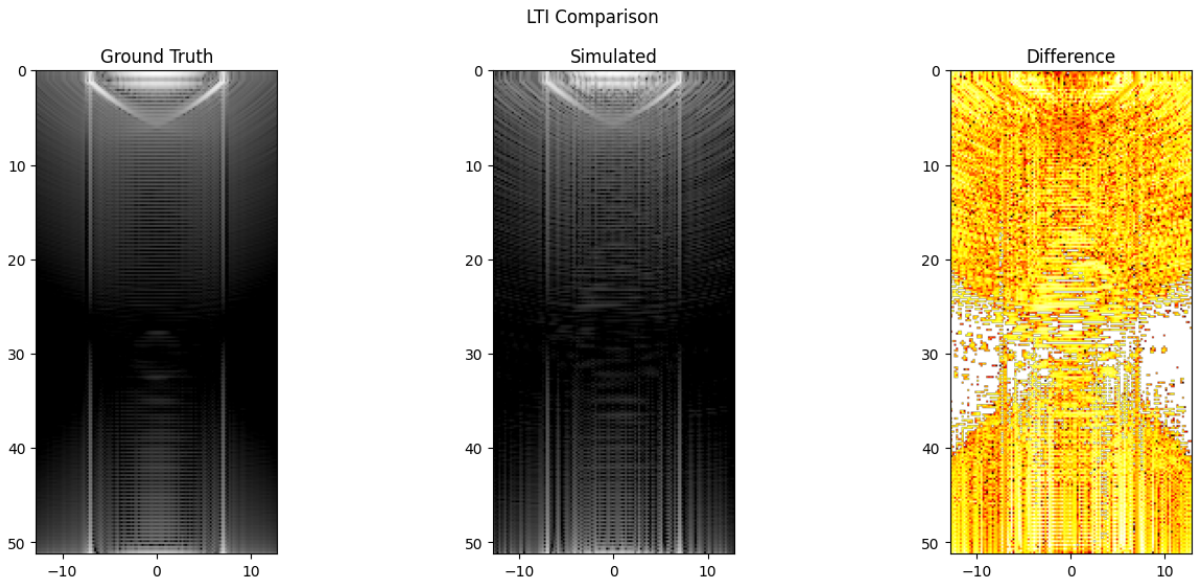

Figure 1: Comparison of the LTI spatial interpolation reconstruction against k-wave model

The hope is that this could potentially reduce latency from minutes to milliseconds, which would then make this DSP learning assignment a reality.

4.2 Mapping Continuous Function Spaces

When first looking at it, this might look like a standard machine learning problem. Many neural networks learn mappings of the form

$$f : \mathbb{R}^n \rightarrow \mathbb{R}^m$$

However, acoustic propagation is not fundamentally a vector-to-vector mapping.

Since the propagation is inherently governed by a partial differential equation, where we take the field of TX inputs and output a field of RX outputs.

Thus, the TX $\rightarrow$ RX propagation is not just a vector-to-vector mapping, rather, we are looking at function spaces to function spaces. The mapping of any TX $\rightarrow$ RX will be consistent no matter what the TX input is. However, if what the mapping is conditioned on (i.e., the state) changes, then the mapping changes.

Because of this, the correct tool for the job is to not just have an NN approximate a function, rather we want to approximate an operator.

This makes a lot of sense. For example, when we look at the LTI case, we don't try to iterate in time to solve. Rather, we use a transfer function, and we can think of the process of applying this transfer function as an operator. This makes sense to use an operator for this situation.

Now, to get into the math of it all:

Let's start with the function space representing the medium

$$\mathcal{A} = (c(x, z), \rho(x, z), M(x, z))$$

And the excitation space

$$\mathcal{E}$$

which represents the transmit pulse and steering parameters.

Our goal is to learn a neural operator

$$G_\theta : \mathcal{A} \times \mathcal{E} \to \mathcal{Y}$$

where $\mathcal{Y}$ is the space of received signals

$$y(x, t)$$

In other words, we want the network to approximate the forward propagation operator of the acoustic PDE.

4.3 Image Reconstruction & Phase

Now, we have some constraints for the forward operator evaluation. Ultrasound RX estimation needs phase accuracy.

This is directly contrasting with many other machine learning tasks, where capturing the general magnitude structure of a signal is sufficient. In this, small phase errors often do not matter.

But, ultrasound imaging is very different as the raw RF channel data is not directly useful. Rather, to form an image, we need to apply Delay-and-Sum (DAS) beamforming.

This is mathematically computed as the following:

For a given focal point

$$\mathbf{r}$$

the beamformed intensity is computed as

$$I(\mathbf{r}) = \sum_{i=1}^{N_c} w_i \, y_i\big(t - \tau_i(\mathbf{r})\big)$$

where $y_i(t)$ is the received signal from channel i, $\tau_i(\mathbf{r})$ is the geometric time delay, and w_i are apodization weights that account for probe geometry.

A time delay corresponds to a phase shift in the frequency domain:

$$\mathcal{F}\{y(t - \tau)\} = Y(\omega)e^{-j\omega\tau}$$

Because of this relationship, even small phase errors can cause destructive interference during beamforming. For example, a half-wavelength error at 4 MHz corresponds to roughly

$$0.2 \text{ mm}$$

in tissue.

To emphasize this even further, a half-wavelength at 4 MHz corresponds to 125 nanoseconds in time.

And if the phase is wrong, signals from different channels cancel instead of strengthening, which can cause a beamformed image to collapse and what we see is nonsensical.

Because of this, a neural approximation for ultrasound must preserve nanosecond-level phase alignment. This requirement strongly constrains which neural architectures will work.

5 Fourier Neural Operator

5.1 Input and Output Parameterization

Before we start building our model, we need to define input and output spaces.

The inputs consist of two physical components: the transmit excitation and the spatial medium.

5.1.1 Excitation Representation

The transmit event is defined by

$$s(t)$$

and a steering angle

$$\theta$$

For a Uniform Linear Array with element positions x_i, the transmit delay applied to the i-th element is

$$\Delta t_i = \frac{x_i \sin(\theta)}{c_0}$$

This analytic relationship allows us to encode beam steering without explicitly passing all 64 transmit waveforms.

5.1.2 Medium Representation

The medium is represented by the spatial maps

$$(c(x,z), \rho(x,z), M(x,z))$$

These maps are passed through a lightweight convolutional encoder to extract spatial features before entering the operator network.

The convolutional encoder has 3 hidden layers, with a kernel size of 64, and 128 filters.

5.1.3 Output Representation

The model predicts the receive time series

$$r_i(t)$$

for each of the 64 array elements.

5.2 The Fourier Neural Operator

The first architecture we decided to try was the Fourier Neural Operator.

The key idea of this is to perform convolution in the frequency domain rather than directly in space. An LTI transfer function essentially just adds various gains on each frequency bin, so generalizing this further for nonlinear systems, the FNO seems to be a good idea.

For a hidden representation

$$v(x)$$

an FNO layer computes

$$(Kv)(x) = \mathcal{F}^{-1}\left\{R_\phi(k)\mathcal{F}\{v\}(k)\right\}$$

where k represents Fourier modes and $R_\phi(k)$ is a learned complex weight tensor.

The layer update becomes

$$v_{t+1}(x) = \sigma\left(Wv_t(x) + (Kv_t)(x)\right)$$

So initially, I hoped that this formulation would allow the model to learn the acoustic propagation patterns.

5.3 Fourier Mode Resolution Requirements

When you go into the math of it, the efficiency of the FNO and the fact that it works in linear time comes from truncating high-frequency Fourier modes.

But, to represent a wave correctly, the model must retain enough modes to resolve the spatial wavelength

$$\lambda = \frac{c_0}{f}$$

So let's start by exploring this idea a little bit more:

If the domain size is L, then the number of required modes is approximately

$$k_{\text{modes}} \geq \frac{L}{\lambda}$$

For an imaging depth

$$L = 0.04 \text{ m}$$

and tissue sound speed

$$c_0 = 1540 \text{ m/s}$$

a 2 MHz pulse produces

$$\lambda = 0.77 \text{ mm}$$

requiring about

$$k_{\text{modes}} \approx 52$$

modes.

This initially seemed pretty manageable to me. However, there is so much more information at higher harmonics in this case. This is just the nature of the problem. Because of this, up to 16 MHz content is still relevant. This means that we need to store over 400 modes. This is just not feasible as a 400x400 matrix just blows up the amount of VRAM and makes training infeasible as well.

6 IQ Modulation To Reduce Computational Complexity

6.1 Complex Baseband Representation

Let's first start looking at a concept used for signal decimation to get more representative samples. This is the idea of IQ modulation. I stands for in-phase and Q stands for quadrature. This just means that we keep in mind the carrier freq of the RF pulse, and demix that carrier frequency, so we just look at the lower frequency envelope of the data.

From there, we can decimate.

In somewhat more rigorous and less wavy terms, we convert the RF signal

$$s_{rf}(t)$$

to a complex baseband representation.

The in-phase component is

$$I(t) = \text{LPF}\{s_{rf}(t) \cdot 2\cos(2\pi f_0 t)\}$$

The quadrature component is

$$Q(t) = \text{LPF}\{s_{rf}(t) \cdot (-2\sin(2\pi f_0 t))\}$$

The complex signal becomes

$$s_{IQ}(t) = I(t) + jQ(t)$$

which represents the slowly varying envelope.

6.2 RF Reconstruction

We can reconstruct the signal from these IQ values using:

$$\hat{s}_{rf}(t) = \hat{I}(t)\cos(2\pi f_0 t) - \hat{Q}(t)\sin(2\pi f_0 t)$$

where the instantaneous phase of the signal is

$$\phi_{IQ}(t) = \arctan\left(\frac{Q(t)}{I(t)}\right)$$

Because of this, the idea was that we can IQ modulate our signal, and use drastically fewer modes for the FNO. That way, it becomes computationally feasible to map TX_IQ $\rightarrow$ RX_IQ using an FNO.

6.3 Loss Functions for Phase-Sensitive Learning

Now, that we have set up the problem, let's talk about the training. We experimented with several loss functions.

For Mean Squared Error (MSE), the loss is defined as:

$$L_{MSE} = \frac{1}{N} \sum_t |\hat{y}(t) - y(t)|^2$$

The main advantages of MSE are that it is simple, stable, and works well for magnitude regression. However, its primary drawbacks are that it does not enforce phase alignment, meaning beamformed images degrade significantly despite a low overall loss.

For Spectral Loss, the equation is:

$$L_{spec} = \sum_\omega |\hat{Y}(\omega) - Y(\omega)|^2$$

This loss is beneficial because it encourages frequency-domain consistency and captures signal structure better than time-domain MSE. On the downside, it still mixes amplitude and phase, leaving phase errors largely uncontrolled.

For Phase Loss, the formula is:

$$L_{phase} = \frac{1}{N} \sum_\omega |\angle\hat{Y}(\omega) - \angle Y(\omega)|$$

The clear advantage here is that it directly penalizes phase error, which better aligns signals for beamforming. Unfortunately, it is highly unstable in areas where signal magnitude is small, causing noise regions to dominate the gradients.

For Amplitude-Weighted Phase Loss, the function is:

$$L_{weighted_phase} = \frac{\sum_\omega |Y(\omega)|\,|\angle\hat{Y}(\omega) - \angle Y(\omega)|}{\sum_\omega |Y(\omega)|}$$

This approach solves many issues by focusing learning on frequencies with strong signal energy, reducing noise sensitivity, and preserving beamforming coherence. However, it is slightly more complex to compute and still requires careful tuning to maintain training stability.

As with all loss things, our final choice was a balance. We incorporated a mix:

$$L_{total} = \lambda_{weighted_phase} L_{weighted_phase} + \lambda_{spec} L_{spec} + \lambda_{mse} L_{mse}$$

where:

$$\lambda_{weighted_phase} = 0.8, \quad \lambda_{spec} = 0.1, \quad \lambda_{mse} = 0.1$$

7 Limitations of the FNO

After all this writing praising the FNO, unfortunately, it was not able to properly simulate ultrasound.

In other words, the FNO ultimately failed to outperform classical LTI models.

To see if the FNO could work if we don't have any complex sorts of inclusions and just work on normal acoustic propagation, we trained it on a dataset of 800 simulations where only the medium maps

$$(c(x,z), \rho(x,z), M(x,z))$$

varied.

Even under this constrained setting, the model struggled to preserve phase accurately across all 64 receive channels. The average magnitude weighted phase loss hovered around 3-5 degrees. The LTI system compared to the k-Wave for reference had an average phase loss of <0.1 degrees.

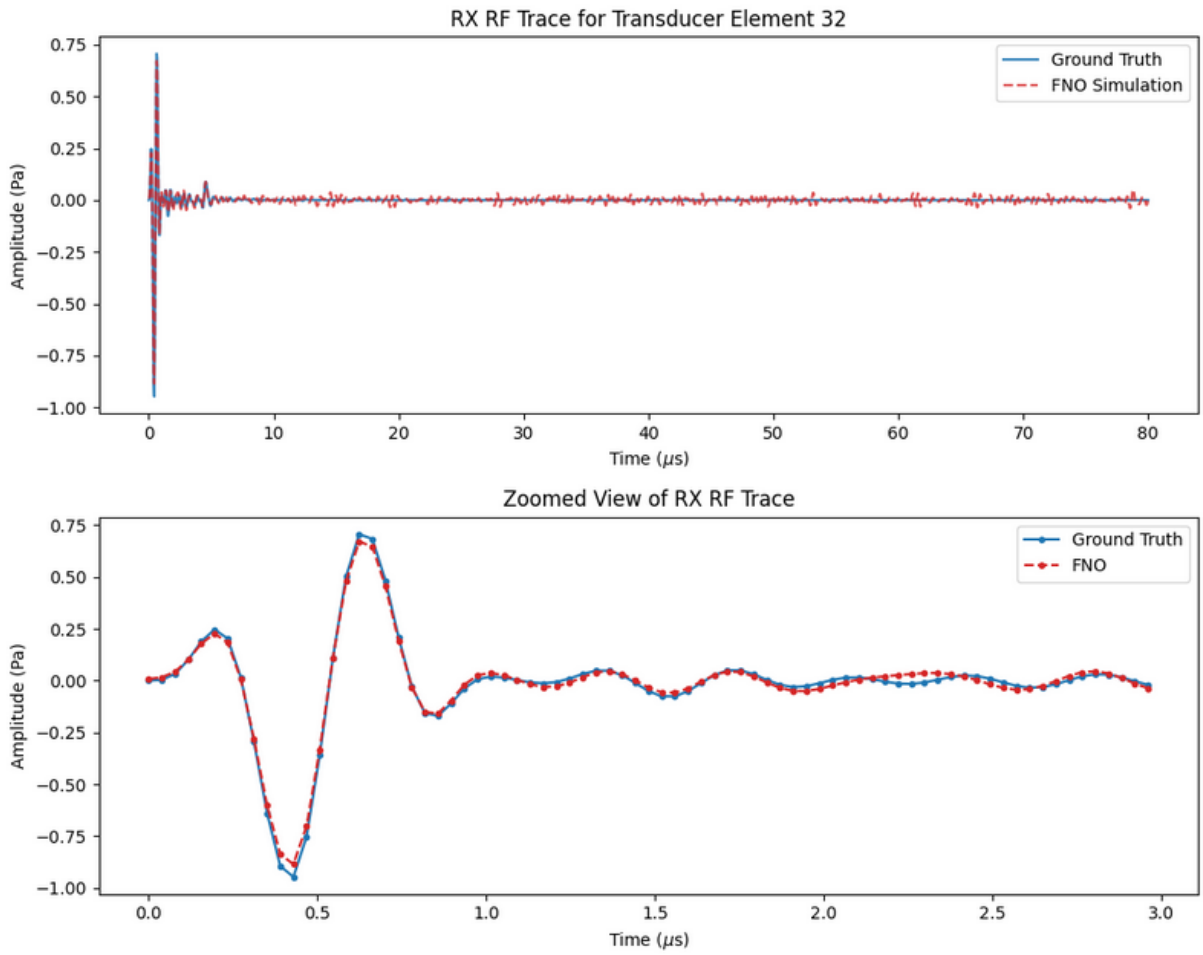

Figure 2: Analysis of a single channel's FNO approximation vs ground truth. The FNO does a very good job at getting a similar signal in the time domain.

The FNO did a good job at approximating the signal; however, there still is a small phase difference between the ground truth.

Because DAS beamforming depends on sub-wavelength phase coherence, small prediction errors led to destructive interference and this was pretty catastrophic.

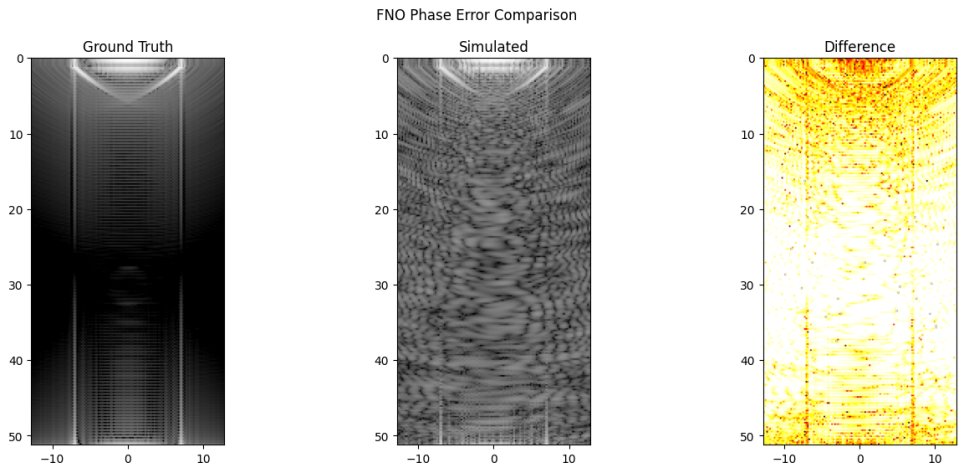

Figure 3: Analysis of Ground Truth vs FNO generated ultrasound images

8 The Wavelet Neural Operator

8.1 Motivation

At this point we've outlined the main issue with the Fourier Neural Operator fairly well. But, I'm stubborn and don't want to quit so I spend some time thinking about Operators outside the Fourier transform.

I was thinking: The FNO is built in the Fourier domain. That means it represents signals using sinusoids, which makes it best for a transient. However, what we are looking at is an RF pulse followed by its echoes. We have a short high-frequency transient where most of the energy is concentrated at, followed by time-delayed and amplitude-attenuated echoes.

Also, we have sharp boundaries in the medium, like tissue interfaces, cyst boundaries, or strong impedance discontinuities. Those reflections can also create wavefronts with sharp temporal and spatial structure.

And this is a well-documented limitation of Fourier analysis.

To get around this problem, I need a representation that is better matched to localized transient signals.

For me, the obvious thing to think about is wavelets.

Unlike sine and cosine waves, wavelets are localized in both time and frequency, which make them ideal for the RF pulse type structure we need in ultrasound.

A wavelet family is built from a single mother wavelet

$$\psi(t)$$

by scaling it and shifting it:

$$\psi_{a,b}(t) = \frac{1}{a}\psi\left(\frac{t-b}{a}\right)$$

where a controls the scale and b controls the position.

Now, looking at the Discrete version, the Discrete Wavelet Transform (DWT) to decompose a signal:

At a high level, the decomposition gives me approximation coefficients, which capture the low-frequency and large-scale structure, alongside detail coefficients, which capture the high-frequency and localized structure.

We can express the decomposition as

$$\mathcal{W}\{v\} = [c_A, c_{D_1}, c_{D_2}, \ldots]$$

where c_A contains the approximation coefficients and $c_{D_1}, c_{D_2}, \ldots$ contain the detail coefficients.

In the WNO case, we can look at this as an operator that acts on each subband separately:

$$R_\phi \cdot \mathcal{W}\{v\} = [R_{c_A} c_A,\ R_{c_{D_1}} c_{D_1},\ R_{c_{D_2}} c_{D_2}, \ldots]$$

where:

$$(Kv)(x) = \mathcal{W}^{-1}\left\{R_\phi \cdot \mathcal{W}\{v\}(x)\right\}$$

This is useful for acoustic propagation in ultrasound as many parts of the acoustic field operate in different scales.

For example, the broad propagation trend through the medium is low-frequency, the exact pulse edge and sharp scattering boundaries are high-frequency, and fine phase structure shows up in localized details.

That is the key difference from the FNO. The FNO more or less discards high-frequency content when it truncates modes. The WNO still compresses the representation, but it does not throw away the localized detail in the same destructive way.

In other words, wavelets let me stay computationally efficient without destroying the part of the signal that carries the phase information. This is so key to my project that involves both low and high-frequency modes.

8.2 Improving Phase

Now let's think about this concept from a phase perspective. All we care about is if the predicted RX data beamforms correctly.

That means that the model needs to preserve the fine timing structure of the wavefront across all channels.

The WNO helps because it separates the problem into scales.

The approximation coefficients can model the slower, large-scale propagation behavior, such as bulk wave travel, low-frequency attenuation trends, and the coarse spatial structure of the medium.

At the same time, the detail coefficients can preserve the sharp transient pulse edges, localized scattering events, and the small timing differences that create phase coherence across channels.

So now it only makes sense that the WNO is a much better match for the ultrasound problem than the FNO.

9 WNO Training and Data Scaling

9.1 First Validation on 800 Samples

Now we need to test it.

To test that, I trained an initial proof-of-concept model on a small dataset of 800 simulated samples, used an 80-sample validation and an 80-sample test set as well.

I kept a lot of constraints just to see if we can learn the wave propagation well. Specifically, the transducer geometry was fixed, the steering angles were fixed, the medium only contained stochastic acoustic speckle, and there were no large structural inclusions.

It worked better than the FNO in this toy setting. We got an average of around 0.5-2 deg of weighted phase loss and we also got a better MSE on this test dataset.

The WNO was able to accurately regress the high-frequency wavelet detail coefficients. After reconstructing the full RF signals and applying Delay-and-Sum (DAS) beamforming, the predicted images matched the k-Wave ground truth much more closely than the FNO outputs did.

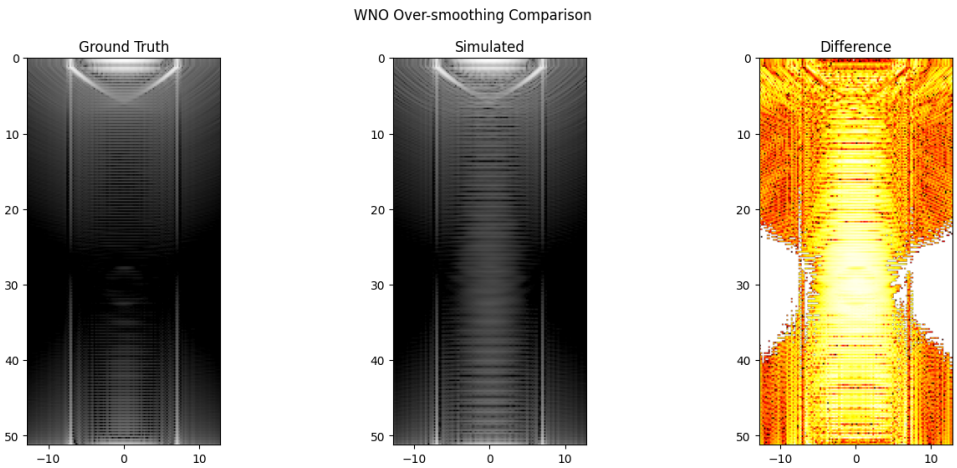

Figure 4: Examination of the Ground Truth Formed Image vs the WNO Formed Image

What is especially interesting is that the time domain signals do not look even close to each other. Since we just trained on amplitude weighted loss, we never really looked at things like magnitude. For this reason, the time domain signals look very different from each other. Perhaps, adding a small MSE component in the future would greatly help the time domain reconstruction look more natural.

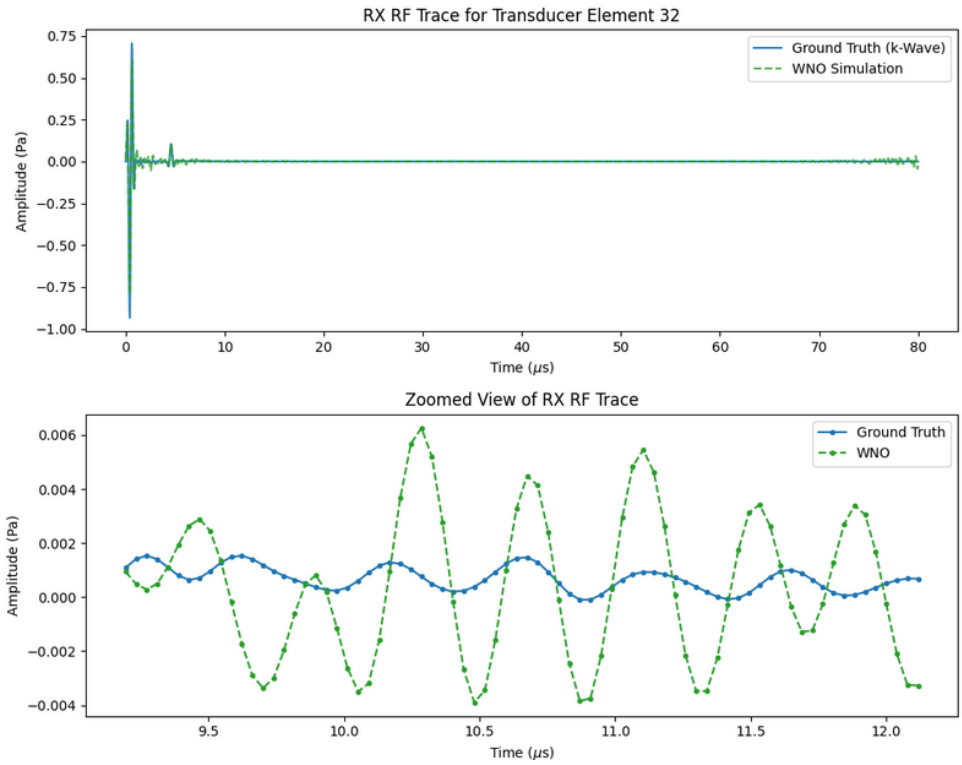

Figure 5: Comparison of WNO vs Ground Truth signals in the time domain. Here, we can see that since the WNO was only trained on phase loss, the ground truth and reconstructed signals look nothing like each other.

We can also look at how the amplitude and phase of the neural operators compare to the LTI approximation of the ground truth as well.

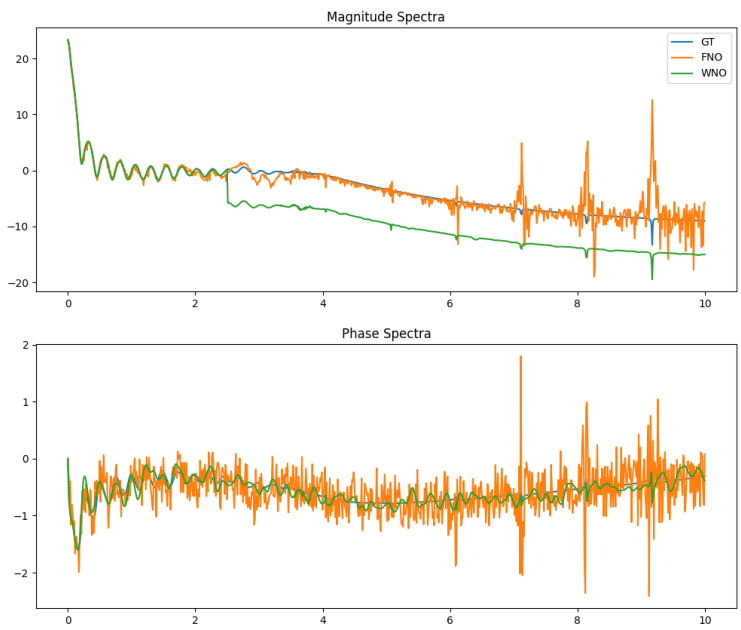

Figure 6: Comparing the magnitude and phase spectra of the FNO, WNO and ground truth LTI approximation. Here we can see that the WNO preserves phase much better than the FNO.

9.2 Scaling Up to Inclusions and 8,000 Samples

I added macroscopic structural inclusions into the medium. These were modeled as anechoic or hyperechoic disks defined by their center position (x, z) and radius r.

This ended up expanding the state space a lot. The inc_mask was updated according to these values. And I generated 8k samples over 2 days to train the data with.

However, the WNO was not able to converge in this case. It really failed pretty spectacularly, giving me over 100 deg of weighted phase loss.

I really believe that this was just from not enough data, as before the propagation worked just fine. Given more data and more training time, I think we would be able to converge.

10 Future Work and Next Steps

Now onto what I would do if I had more time and resources.

First, I would get more data, as I think what limited the WNO from working better was the limited dataset. Second, I would make the vision encoder more complicated or use an autoencoder or some other way to compress the information of the state maps and encode them better. I think that the inclusions really threw off the possibilities for state space. Finally, I would quantize, prune, and export the model in C++ using ONNX to test the real-time latency and see if this architecture can infer fast enough for the actual DSP project.

11 Conclusion

At the beginning of this project, the main goal was pretty simple, I wanted to get around the huge computational cost of the k-Wave pseudo-spectral solver so that ultrasound simulation could actually run in real time for the Applied DSP class that I am creating. My hope was that I was able to create a approximate solver that could help give the students an interactive simulation software to experiment with live.

Since there are some nonlinearities in ultrasound due to fluids in tissue, I wanted to use a nonlinear simulation tool. Since k-Wave and other iterate through time solutions were too costly, I explored neural operator methods, mainly the Fourier Neural Operator (FNO) and the Wavelet Neural Operator (WNO). The idea was that instead of stepping through the acoustic PDE one timestep at a time, the model could learn the global forward map from the medium and transmit excitation directly to the received RF data.

What ended up being most interesting was not just whether the models worked, but *how* they failed.

The FNO looked promising at first because it drove the Mean Squared Error (MSE) very low, lower than the LTI model even. The FNO did a decent job learning the broad energy envelope of the wavefield after using IQ modulation/demodulation to help focus on the lower freq envolope without drastically inceasing compuational complexity. However, its problem was that ultrasound imaging does not just care about the envelope, rahter it also cares about phase coherence. Delay-and-Sum (DAS) beamforming depends on extremely precise phase alignment across channels. Since the FNO has small phase inconsitancies, the algorithm quickly fell apart in image reconstruction. Even though the MSE loss looked good, the actual image quality collapsed because the channels no longer interfered constructively.

To fix this, I tried the WNO. Since wavelets are much better at representing localized transients, I expected the WNO to better preserve the kind of fine-scale structure and better localize the phase. To some extent it did. The reconstrcuted image looked much better, however, it's time domain signal looked very off from from the k-wave rx signal.

And that is really where the main conclusion of the project comes from.

In the end, the most reliable solution was not the neural network at all. It was the classical Linear Time-Invariant (LTI) lookup table. It shows that the assumptions that we make using physical properties actually make sense.

For a static tissue medium, and without pushing the simulation into strongly nonlinear regimes, the propagation behavior is dominated by linear scattering and linear time delays. That means the system is already very well described by an LTI model. Once that is true, the cleanest thing to do is not to approximate the physics with a learned black box. It is to directly exploit the linear structure that is already there.

At the end of the day, I am a bit disappointed all this work got outshines by a very simple method. However, I did learn a lot and think that this project really helps my portfolio, especially as an undergrad.